



CoffeeDelim

This project may appeal to programmers who dislike the indentation-based structure in CoffeeScript but otherwise like the language for its simplicity, terseness and expressiveness.

Although CoffeeScript was influenced by Python and Ruby, it uses neither Python's colon nor Ruby's **end** keyword. **CoffeeDelim** reintroduces both to clearly delimit code blocks.

- A function block is opened with a trailing arrow as usual and is terminated with the **end** keyword.
- Any other block is opened with a trailing colon and is also terminated with the **end** keyword. The sole exception is the **switch** construct that must be terminated with **end switch**.
- A conditional or iteration one-liner use the inline **then** keyword as usual. Not a colon like in Python. See 'General syntax' section for more details.

CoffeeDelim offers some advantages:

- Whitespace is no longer significant.
- Some people feel more comfortable with the structural integrity of delimited blocks.
- Safe copy-pasting without worrying about indentation levels.
- Safe auto-formatting.

Syntax restrictions

CoffeeDelim is a text preprocessor without a stack or a lexer. It works mostly on a line-by-line basis. As a result, it has a few syntax restrictions, which are listed below. Fortunately, they align with recommended coding practices and help improve readability and consistency.

- No comment is allowed after a trailing colon or a trailing arrow. The preprocessor having no lexer, there is no reliable way to strip the comment, even with a complex regex, and the comment would prevent a colon or an arrow detection.

Example:

```
myfunction = () ->      #comment not allowed here
  code
end
```

- Multi-line heredocs and block comments must stand on their own lines. This way, the preprocessor can correctly identify the structure. Otherwise if the text contains any trailing colons or trailing arrows, or leading keywords like **end**, it would open or close false blocks.

Example:

```
variable =
  """ this is a heredoc
  spanning several lines
  of text."""
```

- Multi-line **if**, **try**, and **switch** constructs must use code blocks only.

Usage

CoffeeDelim outputs to *stdout*.

The option *--format* reindents lines consistently, the code remains unchanged.

node <path>**coffeedelim** [*--format*] <script>.coffee

Typical usage

The CoffeeScript compiler must be installed to get JavaScript.

Convert and compile to JavaScript in one step

node <path>**coffeedelim** script.coffee | **coffee -s -c** > script.js

Get a converted copy of a source

node <path>**coffeedelim** script.coffee > script.converted.coffee

Get a formatted copy of a source

node <path>**coffeedelim --format** script.coffee > script.formatted.coffee

Typical usage with Geany code editor

The following command line can be added to the Build menu when using Geany in Linux.

It converts and compiles in one step.

If a conversion or compilation error occurs, it shows the error in the Message window, and the cursor can be jumped directly to the guilty line(s) using “Next Error” and “Previous Error” available in the same Build menu.

```
set -o pipefail; node <path>coffeedelim "%f" | coffee -s -c >
"%e".js 2> >(sed 's/[stdin]/%f/' >&2)
```

General syntax

```
functionName = (parms) ->  
  code  
end
```

```
functionName = (parms) -> code
```

```
do ->  
  code  
end
```

```
do -> code
```

```
[variable =] if <condition> :  
  code  
else if <condition> :  
  code  
else :  
  code  
end
```

```
[variable =] if <condition> then code [else code]
```

```
code if <condition>
```

```
[variable =] while <condition> [guard clauses] :  
  code  
end
```

```
[variable =] while <condition> [guard clauses] then code
```

```
[variable =] for <iterable> [guard clauses] :  
  code  
end
```

```
[variable =] for <iterable> [guard clauses] then code
```

```
[variable =] switch [variable] :  
  when <condition> :  
    code  
  [when <condition> :  
    code]  
  [else :  
    code]  
end switch
```

```
try :  
  code  
[catch [var] :  
  code]  
[finally :  
  code]  
end
```

```
class name :  
  code  
end
```